Homework 2

Aldo Nunez

2026-10-17

```
library(tidyverse)
library(tidymodels)
library(kknn)
tidymodels_prefer()

set.seed(231)

abalone <- read_csv("data/abalone.csv")
```

Linear Regression and KNN

For this assignment, we will be working with a data set from the UCI (University of California, Irvine) Machine Learning repository (see website here). The full data set consists of 4,177 observations of abalone in Tasmania. (Fun fact: Tasmania supplies about 25% of the yearly world abalone harvest.)

The age of an abalone is typically determined by cutting the shell open and counting the number of rings with a microscope. The purpose of this data set is to determine whether abalone age (**number of rings + 1.5**) can be accurately predicted using other, easier-to-obtain information about the abalone.

The full abalone data set is located in the `\data` subdirectory. Read it into *R* using `read_csv()`. Take a moment to read through the codebook (`abalone_codebook.txt`) and familiarize yourself with the variable definitions.

Make sure you load the `tidyverse` and `tidymodels`!

Question 1

Your goal is to predict abalone age, which is calculated as the number of rings plus 1.5. Notice there currently is no `age` variable in the data set. Add `age` to the data set.

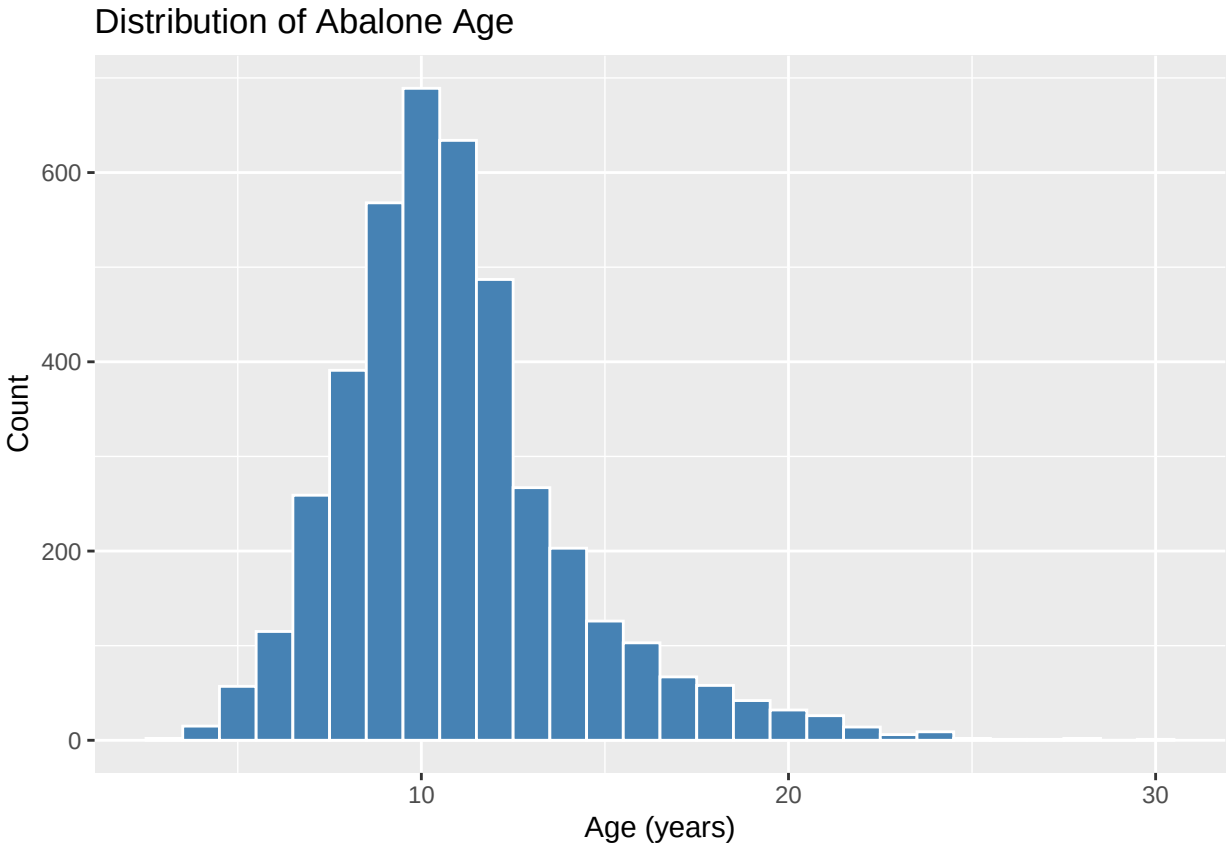
Assess and describe the distribution of `age`.

```
abalone <- abalone %>%
  mutate(age = rings + 1.5)

summary(abalone$age)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    2.50    9.50   10.50   11.43   12.50   30.50
```

```
abalone %>%
  ggplot(aes(x = age)) +
  geom_histogram(binwidth = 1, fill = "steelblue", color = "white") +
  labs(title = "Distribution of Abalone Age", x = "Age (years)", y = "Count")
```



Age goes from 2.5 to 30.5 years. The mean is about 11.4 and the median is 10.5, with a standard deviation of about 3.2. The distribution is unimodal and **right-skewed** (skewness ≈ 1.1). Most abalone are between about 7.5 and 17.5 years old (the 5th to 95th percentiles), and the peak is around 9–11 years. There is a long right tail with a few abalone older than 20, while there are very few young abalone (under about 5). Because the mean is higher than the median, the older abalone in the tail pull the mean up.

Question 2

Split the abalone data into a training set and a testing set. Use stratified sampling. You should decide on appropriate percentages for splitting the data.

Remember that you'll need to set a seed at the beginning of the document to reproduce your results.

```
abalone_split <- initial_split(abalone, prop = 0.75, strata = age)
abalone_train <- training(abalone_split)
abalone_test  <- testing(abalone_split)

dim(abalone_train)
```

```
## [1] 3131   10
```

```
dim(abalone_test)
```

```
## [1] 1046   10
```

I used a 75/25 train/test split, stratified on `age`. The seed is set at the top of the document (`set.seed(231)`). With about 4,200 observations, 75% gives plenty of data to fit the models, and the remaining ~1,000 observations are enough to get a reliable estimate of test performance.

Question 3

Using the **training** data, create a recipe predicting the outcome variable, `age`, with all other predictor variables. Note that you **should not** include `rings` to predict `age`. *Explain why you shouldn't use* ***`rings`*** *to predict* ***`age`***.

Steps for your recipe:

1. dummy code any categorical predictors
2. create interactions between
    - `type` and `shucked_weight`,
    - `longest_shell` and `diameter`,
    - `shucked_weight` and `shell_weight`
3. center all predictors, and
4. scale all predictors.

You'll need to investigate the `tidymodels` documentation to find the appropriate step functions to use.

We shouldn't use `rings` to predict `age` because `age` is defined as `rings + 1.5`, so `rings` gives away the answer exactly. Also, the point of the problem is to predict age *without* counting rings, since counting them means cutting the shell open and using a microscope. If we already had `rings`, there would be nothing to predict.

```
abalone_recipe <- recipe(age ~ ., data = abalone_train %>% select(-rings)) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_interact(terms = ~ starts_with("type"):shucked_weight +
                  longest_shell:diameter +
                  shucked_weight:shell_weight) %>%
  step_center(all_predictors()) %>%
  step_scale(all_predictors())

tidy(abalone_recipe)
```

```
## # A tibble: 4 x 6
##   number operation type     trained skip  id
##    <int> <chr>     <chr>    <lgl>   <lgl> <chr>
## 1      1 step      dummy    FALSE   FALSE dummy_TyI4K
## 2      2 step      interact FALSE   FALSE interact_0S2j2
## 3      3 step      center   FALSE   FALSE center_2O5hA
## 4      4 step      scale    FALSE   FALSE scale_VyfLx
```

Question 4

Create and store a linear regression object using the `"lm"` engine.

```
lm_model <- linear_reg() %>%
  set_engine("lm")
```

Question 5

Create and store a KNN object using the `"kknn"` engine. Specify `k = 7`.

```
knn_model <- nearest_neighbor(neighbors = 7) %>%
  set_engine("kknn") %>%
  set_mode("regression")
```

Question 6

Now, for each of these models (linear regression and KNN):

1. set up an empty workflow,
2. add the model, and
3. add the recipe that you created in Question 3.

Note that you should be setting up two separate workflows.

Fit both models to the training set.

```
lm_wflow <- workflow() %>%
  add_model(lm_model) %>%
  add_recipe(abalone_recipe)

knn_wflow <- workflow() %>%
  add_model(knn_model) %>%
  add_recipe(abalone_recipe)

lm_fit  <- fit(lm_wflow, abalone_train)
knn_fit <- fit(knn_wflow, abalone_train)

lm_fit %>%
  extract_fit_parsnip() %>%
  tidy()
```

```
## # A tibble: 14 x 5
##    term                           estimate std.error statistic  p.value
##    <chr>                             <dbl>     <dbl>     <dbl>    <dbl>
##  1 (Intercept)                      11.4      0.0380   301.    0
##  2 longest_shell                     0.421    0.285      1.48  1.40e- 1
##  3 diameter                          1.88     0.313      6.02  1.93e- 9
##  4 height                            0.206    0.0703     2.92  3.48e- 3
##  5 whole_weight                      4.92     0.382     12.9   5.60e-37
##  6 shucked_weight                   -4.30     0.250    -17.2   1.54e-63
##  7 viscera_weight                   -0.810    0.157     -5.17  2.48e- 7
##  8 shell_weight                      1.62     0.212      7.63  3.12e-14
##  9 type_I                           -1.01     0.117     -8.61  1.10e-17
## 10 type_M                           -0.344    0.107     -3.22  1.31e- 3
## 11 type_I_x_shucked_weight           0.549    0.0884     6.20  6.20e-10
## 12 type_M_x_shucked_weight           0.353    0.115      3.07  2.15e- 3
## 13 longest_shell_x_diameter         -2.50     0.409     -6.10  1.19e- 9
## 14 shucked_weight_x_shell_weight    -0.188    0.206     -0.913 3.61e- 1
```

Question 7

Use your linear regression `fit()` object to predict the age of a hypothetical female abalone with longest_shell = 0.50, diameter = 0.10, height = 0.30, whole_weight = 4, shucked_weight = 1, viscera_weight = 2, and shell_weight = 1.

```
new_abalone <- tibble(type = "F", longest_shell = 0.50, diameter = 0.10,
                      height = 0.30, whole_weight = 4, shucked_weight = 1,
                      viscera_weight = 2, shell_weight = 1)

predict(lm_fit, new_data = new_abalone)
```

```
## # A tibble: 1 x 1
```

```
##   .pred
##   <dbl>
## 1  24.9
```

The linear regression model predicts that this abalone is about **24.9 years old**. I wouldn't trust this number much, though. The hypothetical abalone is far outside the range of the data: its `whole_weight` of 4 is above the largest value in the data set (about 2.83), and a `diameter` of 0.10 with a `height` of 0.30 is a shape that real abalone don't have. So this prediction is an extrapolation.

Question 8

Now you want to assess your models' performance. To do this, use the `yardstick` package:

1. Create a metric set that includes R^2, RMSE (root mean squared error), and MAE (mean absolute error).
2. Use `augment()` to create a tibble of your model's predicted values from the **testing data** along with the actual observed ages (these are needed to assess your model's performance).
3. Finally, apply your metric set to the tibble, report the results, and interpret the *R^2* value.

Repeat these steps once for the linear regression model and for the KNN model.

```
abalone_metrics <- metric_set(rsq, rmse, mae)

lm_test_res <- augment(lm_fit, new_data = abalone_test) %>%
  select(age, .pred)
abalone_metrics(lm_test_res, truth = age, estimate = .pred)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rsq     standard       0.524
## 2 rmse    standard       2.23
## 3 mae     standard       1.62
```

```
knn_test_res <- augment(knn_fit, new_data = abalone_test) %>%
  select(age, .pred)
abalone_metrics(knn_test_res, truth = age, estimate = .pred)
```

```
## # A tibble: 3 x 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rsq     standard       0.483
## 2 rmse    standard       2.33
## 3 mae     standard       1.63
```

Model	R^2	RMSE	MAE
Linear regression	0.524	2.23	1.62
KNN (k = 7)	0.483	2.33	1.63

Linear regression: The R^2 of 0.524 means that about 52% of the variation in abalone age in the testing set is explained by the model. The other ~48% is not explained. Predictions are off by about 2.2 years on average (RMSE), and the typical absolute error is about 1.6 years (MAE).

KNN: The R^2 of 0.483 means that about 48% of the variation in age in the testing set is explained by the KNN model. Its RMSE is 2.33 years and its MAE is 1.63 years.

For both models, a good amount of the variation in age is still left unexplained, so these physical measurements only predict age moderately well.

Question 9

Which model performed better on the testing data? Explain why you think this might be. Are you surprised by any of your results? Why or why not?

The **linear regression model did better** on the testing data. It has a higher R^2 (0.524 vs. 0.483) and a lower RMSE (2.23 vs. 2.33). The two models have almost the same MAE (1.62 vs. 1.63), so the difference between them is small.

I think this is because of the bias-variance tradeoff. KNN with k = 7 is very flexible. Each prediction is just the average of 7 nearby training points, so it has low bias but high variance. You can see it overfit: on the training set, KNN has an R^2 of 0.81 and an RMSE of 1.45, but on the test set those drop to 0.48 and 2.33. The linear model's training and test results are close (training R^2 0.57 vs. test 0.52), so it generalizes better. The relationship between the size/weight measurements and age is also mostly monotone, so a linear model with a few interaction terms already captures most of it. KNN also has a harder time because there are many predictors (13 after dummy coding and adding interactions), and nearest-neighbor methods get worse as the number of dimensions grows.

I was a little surprised that the simpler linear model beat KNN, since KNN can fit non-linear patterns. But given how much KNN overfit the training data with a small k, it makes sense. I was also somewhat surprised that both models only explain about half of the variation in age. Abalone that are the same size can be very different ages (growth probably slows down once they're older), so size and weight measurements alone can't fully predict age.

Required for 231 Students

In lecture, we presented the general bias-variance tradeoff, which takes the form:

$$E[(y_0 - \hat{f}(x_0))^2] = Var(\hat{f}(x_0)) + [Bias(\hat{f}(x_0))]^2 + Var(\epsilon)$$

where the underlying model $Y = f(X) + \epsilon$ satisfies the following:

- ϵ is a zero-mean random noise term and X is non-random (all randomness in Y comes from ϵ);
- (x_0, y_0) represents a test observation, independent of the training set, drawn from the same model;
- $\hat{f}(.)$ is the estimate of f obtained from the training set.

Question 10 Which term(s) in the bias-variance tradeoff above represent the reducible error? Which term(s) represent the irreducible error?

The reducible error is $Var(\hat{f}(x_0)) + [Bias(\hat{f}(x_0))]^2$. These two terms depend on how we estimate f, so we can make them smaller by picking a better model or method.

The irreducible error is $Var(\epsilon)$. It comes from the random noise in Y, so no matter how good our estimate $\hat{f}$ is, we can't get rid of it.

Question 11 Using the bias-variance tradeoff above, demonstrate that the expected test error is always at least as large as the irreducible error.

A variance is always non-negative, so $Var(\hat{f}(x_0)) \geq 0$, and a square is always non-negative, so $[Bias(\hat{f}(x_0))]^2 \geq 0$. That means

$$E[(y_0 - \hat{f}(x_0))^2] = \underbrace{Var(\hat{f}(x_0))}_{\geq 0} + \underbrace{[Bias(\hat{f}(x_0))]^2}_{\geq 0} + Var(\epsilon) \geq Var(\epsilon).$$

The expected test error equals $Var(\epsilon)$ only when $\hat{f}(x_0)$ has zero variance and zero bias, which means $\hat{f}(x_0) = f(x_0)$ every time.

Question 12 Prove the bias-variance tradeoff.

Hints:

- use the definition of $Bias(\hat{f}(x_0)) = E[\hat{f}(x_0)] - f(x_0)$;
- reorganize terms in the expected test error by adding and subtracting $E[\hat{f}(x_0)]$

Since x_0 is non-random, $f(x_0)$ is a constant. The test point is $y_0 = f(x_0) + \epsilon_0$ with $E[\epsilon_0] = 0$ and $Var(\epsilon_0) = E[\epsilon_0^2] = Var(\epsilon)$. Because (x_0, y_0) is independent of the training set, ϵ_0 is independent of $\hat{f}(x_0)$.

Step 1: Separate out the noise.

$$\begin{aligned} E[(y_0 - \hat{f}(x_0))^2] &= E[(f(x_0) + \epsilon_0 - \hat{f}(x_0))^2] \\ &= E[(f(x_0) - \hat{f}(x_0))^2] + 2E[\epsilon_0(f(x_0) - \hat{f}(x_0))] + E[\epsilon_0^2]. \end{aligned}$$

By independence, $E[\epsilon_0(f(x_0) - \hat{f}(x_0))] = E[\epsilon_0]\, E[f(x_0) - \hat{f}(x_0)] = 0$. Also $E[\epsilon_0^2] = Var(\epsilon)$. So

$$E[(y_0 - \hat{f}(x_0))^2] = E[(f(x_0) - \hat{f}(x_0))^2] + Var(\epsilon).$$

Step 2: Add and subtract $E[\hat{f}(x_0)]$.

$$\begin{aligned} E[(f(x_0) - \hat{f}(x_0))^2] &= E\Big[\big(\hat{f}(x_0) - E[\hat{f}(x_0)] + E[\hat{f}(x_0)] - f(x_0)\big)^2\Big] \\ &= E\Big[\big(\hat{f}(x_0) - E[\hat{f}(x_0)]\big)^2\Big] + 2\big(E[\hat{f}(x_0)] - f(x_0)\big)\, E\big[\hat{f}(x_0) - E[\hat{f}(x_0)]\big] + \big(E[\hat{f}(x_0)] - f(x_0)\big)^2. \end{aligned}$$

$E[\hat{f}(x_0)] - f(x_0)$ is a constant, so it can come out of the expectation in the middle term. The middle term is then zero, because $E\big[\hat{f}(x_0) - E[\hat{f}(x_0)]\big] = E[\hat{f}(x_0)] - E[\hat{f}(x_0)] = 0$.

The first term is the definition of $Var(\hat{f}(x_0))$, and the last term is $[Bias(\hat{f}(x_0))]^2$ by the definition $Bias(\hat{f}(x_0)) = E[\hat{f}(x_0)] - f(x_0)$. So

$$E[(f(x_0) - \hat{f}(x_0))^2] = Var(\hat{f}(x_0)) + [Bias(\hat{f}(x_0))]^2.$$

Step 3: Put it together.

$$E[(y_0 - \hat{f}(x_0))^2] = Var(\hat{f}(x_0)) + [Bias(\hat{f}(x_0))]^2 + Var(\epsilon). \qquad \blacksquare$$